

Root-free local C build-toolchain obtain — deep research (§11.4.150)

Revision: 1 **Last modified:** 2026-06-29T00:00:00Z **Authority:** vasic-digital tmux project — RESEARCH ARTEFACT (no source edited) **Mandate:** Operator (2026-06-29): "All mandatory dependencies MUST BE available on local level and obtained for project to build and install fully autonomously"

- **HARD constraint:** NO sudo/su, NO user interaction in automation. **Scope:** obtaining a C BUILD TOOLCHAIN (compiler + linker + libc headers + crt objects) AND the autotools tmux's ./configure needs, all LOCAL + ROOT-FREE, on linux x86_64 + linux arm64 + macOS — closing the gap left by the already-shipped scripts/obtain_local_deps.sh (which obtains libevent/ncurses/jemalloc + the Go toolchain locally, but assumes a working host C toolchain).
-

0. Executive summary / RECOMMENDATION

Two cooperating changes close the gap, both root-free + non-interactive:

1. **C toolchain** → **zig cc** obtained as a single prebuilt .tar.xz (kind=toolchain, mirroring the existing Go entry). Zig bundles clang + lld linker + glibc/musl headers + crt objects, is fully relocatable, needs no root, and one project obtains it for ALL four targets (linux x86_64/aarch64, macOS x86_64/aarch64). It is used as CC="<zig> cc" to build **libevent**, **ncurses**, **jemalloc** **AND** **tmux** — because on a bare host the existing source builds of those deps ALSO fail (host cc cannot link), so the toolchain obtain must run FIRST and feed every downstream source build.
2. **Autotools gap** → **AVOID it** by building tmux from the official **3.6a RELEASE tarball** (which ships a pre-generated configure + aclocal.m4 + Makefile.in + cmd-parse.c), NOT from the git-

submodule working tree (which tracks only configure.ac + autogen.sh → would force autoconf/automake/aclocal/autoreconf/bison/pkg.m4). With the release tarball:

- autoconf/automake/aclocal/autoreconf/m4/perl → **NOT needed**;
- bison/byacc/yacc → **NOT needed** (set YACC=true; cmd-parse.c is pre-generated so the no-op is never invoked — proven below);
- pkg-config → **NOT strictly needed** (tmux configure has AC_SEARCH_LIBS fallbacks for both mandatory deps; we pass -I/-L for the local libevent/ncurses, and keep PKG_CONFIG_PATH opportunistically).

Residual host assumptions after both changes (far more commonly present than the missing glibc-devel that triggers the whole problem): make, tar, xz, curl|wget, sha256sum|shasum, awk/sed/sh. These are honest boundaries — see §6. make is the only non-trivial residual; it is in the base of essentially every Linux install and is a separate, smaller follow-on if a truly make-less host ever appears.

macOS: keep the existing Xcode-CLT + Homebrew path as PRIMARY (CLT is present on virtually every Mac that already ran git/curl); document zig cc as a CLT-less fallback flagged **UNCONFIRMED** (could not be live-tested from this Linux host) per §11.4.81 honest-boundary.

1. Repo evidence — the autotools verdict (captured FACTS)

1.1 The submodule tracks NO generated configure

```
$ git submodule status tmux
cc117b5048f77a4842820f8ebbe3a86e5c077224 tmux (3.6a)
$ git -C tmux ls-files configure configure.ac autogen.sh
Makefile.in
autogen.sh
configure.ac          # <-- only these two are TRACKED
```

The tmux/configure, tmux/Makefile.in, tmux/etc/{missing,install-sh, config.guess,...} present in THIS working tree were generated locally (mtime 2026-05-21) by a prior autogen.sh run. **A fresh git submodule update checks out ONLY the tracked files → NO configure exists →** build_native.sh:233 (if [! -f configure]; then sh autogen.sh; fi) runs autogen.sh.

1.2 autogen.sh requires the full autotools generator stack

```
$ cat tmux/autogen.sh
aclocal || die "aclocal failed"
automake --add-missing --force-missing --copy --foreign || die
"automake failed"
autoreconf || die "autoreconf failed"
```

This is the operator's failing log (configure.ac:8: installing 'etc/missing' is automake --add-missing). It needs **aclocal + automake + autoreconf + autoconf + m4 + perl**, AND because configure.ac uses PKG_CHECK_MODULES (below), aclocal needs **pkg.m4** (shipped by pkg-config/pkgconf), AND AC_PROG_YACC/cmd-parse.y means the regenerated build needs **bison/yacc**.

1.3 configure.ac tool requirements

```
3: AC_INIT([tmux], 3.6a)
8: AM_INIT_AUTOMAKE([foreign subdir-objects])
45: AC_PROG_CC
50: AC_PROG_INSTALL
51: AC_PROG_YACC
52: PKG_PROG_PKG_CONFIG
226/238/281/293/306: PKG_CHECK_MODULES(...) # libevent + ncurses
(mandatory)
273-276: AC_CHECK_PROG(found_yacc,$YACC,...); AC_MSG_ERROR("yacc
not found")
```

1.4 The RELEASE tarball ships the generated files (DECISIVE — live-verified)

```
$ curl -fsSL ../releases/download/3.6a/tmux-3.6a.tar.gz #
750698 bytes
$ sha256sum tmux-3.6a.tar.gz
b6d8d9c76585db8ef5fa00d4931902fa4b8cbe8166f528f44fc403961a3f3759
tmux-3.6a.tar.gz
$ tar tzf tmux-3.6a.tar.gz | grep -E '(configure|aclocal.m4|
Makefile.in|cmd-parse.c)$'
tmux-3.6a/configure
tmux-3.6a/aclocal.m4
tmux-3.6a/Makefile.in
tmux-3.6a/cmd-parse.c # <-- the yacc OUTPUT ships pre-
generated
```

Building from this tarball eliminates the entire autotools GENERATOR stack.

1.5 yacc is bypassable with YACC=true (live-verified in the generated configure)

The generated configure from the release tarball:

```
4578: for ac_prog in 'bison -y' byacc          # search list
4588:   ac_cv_prog_YACC="$YACC" # Let the user override the test
<-- KEY
4619: test -n "$YACC" || YACC="yacc"           # default if unset
6081:   as_fn_error $? "\"yacc not found\""    # hard error if
$YACC missing
```

export YACC=true (or /usr/bin/true) → configure accepts it (override path, line 4588), the AC_CHECK_PROG(found_yacc,\$YACC,...) finds true → no error. Because cmd-parse.c ships newer-or-equal to cmd-parse.y in the dist tarball (and we touch cmd-parse.c after extract for safety), make NEVER runs the .y.c rule → the true no-op is never invoked. **No bison/byacc/yacc needed.**

1.6 pkg-config is NOT strictly required (live-verified)

Both mandatory deps fall back to AC_SEARCH_LIBS when pkg-config / the .pc is absent (4th macro arg = found_*=no, so PKG_CHECK_MODULES does NOT hard-error):

```
configure.ac libevent:
PKG_CHECK_MODULES(LIBEVENT_CORE,libevent_core>=2,...,found_libevent
→
PKG_CHECK_MODULES(LIBEVENT,libevent>=2,...,found_libevent=no)
→ AC_SEARCH_LIBS(event_init,[event_core
event event-1.4])
→ AC_CHECK_HEADER(event2/event.h)
configure.ac ncurses:  PKG_CHECK_MODULES(LIBTINFO/LIBNCURSES/
LIBNCURSESW,...,found_ncurses=no)
→ AC_SEARCH_LIBS(setupterm,[tinfo terminfo
ncurses ncursesw])
→ AC_CHECK_HEADER(ncurses.h)
generated configure:  line 5956 "checking for library containing
event_init" ✓ present
```

So passing CPPFLAGS=-I<local-incdir> + LDFLAGS=-L<local-libdir> for the local libevent/ncurses is sufficient even with NO pkg-config binary. (When pkg-config IS present + PKG_CONFIG_PATH points at the local .pc dir — as build_native.sh already arranges — the pkg-config path is used, which is cleaner; both work.)

1.7 Host capability for the .tar.xz zig tarball (this host)

```
$ xz --version → xz (XZ Utils) 5.4.7      $ tar --version → tar
(GNU tar) 1.35
```

GNU tar shells out to xz for .tar.xz. Present here; flagged as a residual host assumption in §6 (zig publishes ONLY .tar.xz for these platforms).

2. Angle 1 — zig cc as the root-free C toolchain (RECOMMENDED)

2.1 What zig provides

zig cc is a clang-based drop-in C/C++ compiler. The single prebuilt tarball bundles: clang frontend + lld linker + glibc headers/stubs (multiple versions) + musl + crt objects + libc++. It is fully relocatable (extract anywhere, run <dir>/zig), needs **no root, no install step, no system libc-dev** (it carries its own). Andrew Kelley's reference article documents it as a "powerful drop-in replacement for gcc/clang", ~45 MiB, all architectures, glibc + musl + Windows.

2.2 Exact version + download URLs (latest STABLE = 0.16.0, released 2026-04-13)

From <https://ziglang.org/download/index.json> (fetched 2026-06-29). URL pattern: <https://ziglang.org/download/<ver>/zig-<arch>-<os>-<ver>.tar.xz> — note zig's tuple order is <arch>-<os> and arch names are x86_64/aarch64 (DIFFERENT from Go's linux-amd64).

platform (uname)	zig tuple	URL
Linux x86_64	x86_64-linux	https://ziglang.org/download/0.16.0/zig-x86_64-linux-0.16.0.tar.xz
Linux aarch64	aarch64-linux	https://ziglang.org/download/0.16.0/zig-aarch64-linux-0.16.0.tar.xz
macOS x86_64	x86_64-macos	https://ziglang.org/download/0.16.0/zig-x86_64-macos-0.16.0.tar.xz
macOS aarch64	aarch64-macos	https://ziglang.org/download/0.16.0/zig-aarch64-macos-0.16.0.tar.xz

2.3 Checksums — WHERE they are authoritatively published (do NOT invent)

Authoritative source: <https://ziglang.org/download/index.json> — each platform entry is { "tarball": "...", "shasum": "<sha256>", "size": <bytes> } under the version key ("0.16.0"). Additionally every tarball has a .minisig (append .minisig to the URL), verifiable against the ZSF minisign public key RWSG0q2NVecA2UPNdBUZykf1CCb147pkmdtYxgb3Ti+J0/wCYvhbAb/U.

sha256 values as reported by index.json on 2026-06-29 (MUST be re-confirmed by the implementer fetching index.json — these were read via a summarising fetch, so treat as provisional until the obtain script re-verifies against index.json/.minisig per §11.4.6):

platform	size (bytes)	shasum (PROVISIONAL — re-verify)
x86_64-linux	55478392	70e49664a74374b48b51e6f3fdfbf437f6395d42509050588bd49abe52
aarch64-linux	51211944	ea4b09bfb22ec6f6c6ceac57ab63efb6b46e17ab08d21f69f3a48b38e
x86_64-macos	57396836	0387557ed1877bc6a2e1802c8391953baddba76081876301c522f5297
aarch64-macos	52238004	b23d70deaa879b5c2d486ed3316f7eaa53e84acf6fc9cc747de152450

Design note: prefer making the obtain mechanism **fetch index.json for the pinned version, parse shasum per platform, and verify the download against it** (belt: also verify .minisig), rather than hardcoding — this removes the transcription-risk above and matches zig's official "always validate the minisign signature" guidance. Hardcoding (Go-entry style) is acceptable only if the values are re-confirmed by downloading index.json at commit time.

2.4 How to invoke (the critical correctness detail)

- Use CC="<zigdir>/zig cc" — the cc SUBCOMMAND, quoted as two words. CC=zig ALONE FAILS autotools: configure's executable-suffix probe passes -o conftest ... and zig reports unknown command: -o (ziglang/zig#12413). The cc subcommand makes zig parse flags clang-style. (The issue thread does not itself demonstrate the 'zig cc' fix; the working CC='zig cc ...' invocation is documented in the cross-compile write-ups cited in §6 and MUST be live-confirmed per §5.)
- Native build (binary runs on the SAME host): do NOT pass -target — let zig detect the host glibc. If portability-to-older-glibc is wanted, target an explicit floor, e.g. -target x86_64-

linux-gnu.2.28 (binary then needs host glibc $\geq$ 2.28). Calibrate on the real host (\$5).

2.5 Known pitfalls (must be carried into the live test)

- `CC=zig` vs `CC='zig cc'` (above) — the #1 autotools breakage.
- `-D_FORTIFY_SOURCE=2` (in `build_native.sh` `CFLAGS`) needs glibc fortify headers + optimisation; with the **glibc** target zig supplies them, but clang may warn/redefine — verify no hard error. (With a musl target FORTIFY is a no-op.)
- `-fstack-protector-strong, -Wl,-z,relro,-z,now, -Wl,--no-as-needed` — all supported by clang + `lld`; verify in the real link.
- ncurses' own build (most complex of the deps) uses awk-generated sources + builds `tic`; zig-cc compatibility **MUST** be live-confirmed.
- Extracted size is large (~200+ MB; tarball ~55 MB) — one-time disk cost in `.local-deps/`.

3. Angle 2 — alternatives (evaluated, NOT recommended; why)

Option	root-free?	single verifiable DL?	cross-platform?	verdict
Bootlin toolchains (toolchains.bootlin.com)	yes	yes (per-arch tarball + a relocate-sdk.sh step)	Linux only (x86_64/aarch64/...); no macOS	Workable for Linux but per-arch tarballs + a relocate step + glibc-version is whatever they built (binary needs $\geq$ that glibc). Zig is simpler (one tarball, all arches, explicit glibc-version targeting).
musl-cross-make / musl.cc prebuilts	yes	prebuilt static musl toolchains	Linux only	A musl-static tmux runs on ANY Linux

Option	root-free?	single verifiable DL?	cross-platform?	verdict
				regardless of host glibc — attractive — but requires rebuilding libevent/ncurses/jemalloc against musl (feasible since we source-build them), and musl.cc has had availability gaps. Zig's bundled -target ... -musl gives the SAME benefit far more reliably. Keep musl as a zig -target OPTION, not a separate downloader. Heavyweight bootstrap (downloader + solver + GB-scale env); conda's relocation is fragile. Overkill vs one zig tarball.
conda/micromamba relocatable gcc	yes-ish	needs micromamba bootstrap + env solve	Linux + macOS	Needs a writable /nix or proot/chroot; too invasive for
nixpkgs portable / nix single-user	needs/nix or proot	no	Linux + macOS	

Option	root-free?	single verifiable DL?	cross-platform?	verdict
				"drop a tarball" autonomy.

glibc-vs-musl note: for a binary that must RUN on the build host, the safest default is glibc matching the host (zig native target). musl-static is the better choice only if the SAME binary must travel across hosts with different glibc — out of scope for the per-host obtain model, but available as a zig -target flip if ever needed.

Conclusion: zig dominates on (one tarball, all 4 targets, root-free, relocatable, glibc-version targeting, bundled libc + lld) — the others each lose on at least one of cross-platform / single-download / root-free / simplicity.

4. Angle 3 — the autotools verdict (NEEDED vs AVOIDABLE)

Build source	autoconf/ automake/ m4/perl	bison/yacc	pkg.m4	pkg-config (binary)
git submodule working tree (no configure)	REQUIRED (autogen.sh)	REQUIRED	REQUIRED	needed (or env- bypass)
3.6a RELEASE tarball (recommended)	NOT needed	NOT needed (YACC=true)	not needed	NOT needed (AC_SEARCH_LIBS)

Verdict: AVOIDABLE. Build tmux from the sha256-pinned 3.6a release tarball (b6d8d9c76585db8ef5fa00d4931902fa4b8cbe8166f528f44fc403961a3f3759) and the entire autotools-GENERATOR stack disappears. This is consistent with how the project already obtains jemalloc/libevent/ncurses/Go as sha256-pinned tarballs. The pinned git submodule stays as the source-of-truth/provenance reference; the release tarball is the SAME 3.6a source with generated files materialised.

(libevent/ncurses/jemalloc release tarballs the project ALREADY downloads likewise ship pre-generated configure → they too need only zig cc + make, no autotools generators.)

5. Angle 4 — macOS path (§11.4.81 honest boundary)

- **PRIMARY (keep as-is):** Xcode Command Line Tools (CLT) clang + Homebrew deps, exactly as build_native.sh Darwin branch does today. CLT is present on effectively every Mac that has already used git/curl (both trigger the CLT prompt on first use), so the "fresh Mac" gap is narrow in practice. NOTE: a truly CLT-less Mac requires `xcode-select --install`, which is **interactive** → violates the no-interaction constraint → cannot be the autonomous path.
 - **FALLBACK (document, flag UNCONFIRMED):** zig cc is cross-platform and can target *-macos; it ships a libSystem.tbd stub so simple libc links work without the full SDK. Whether it links a COMPLETE tmux (ncurses/libevent + system libs) on a CLT-less Mac is **UNCONFIRMED — not live-testable from this Linux host**. Per §11.4.6 this MUST be proven on a real Mac before any claim; until then macOS autonomy rests on CLT being present.
-

6. Angle 5 — concrete integration design (the change-set)

6.1 scripts/obtain_local_deps.sh — add a cc (zig) kind=toolchain dep

Mirror the existing go entry. New registry branches (illustrative):

```
cc:kind)          → toolchain
cc:envprefix)     → CC
cc:version)       → 0.16.0
cc:url_x86_64_linux) → https://ziglang.org/download/0.16.0/zig-x86_64-linux-0.16.0.tar.xz
cc:url_aarch64_linux) → .../zig-aarch64-linux-0.16.0.tar.xz
cc:url_x86_64_darwin) → .../zig-x86_64-macos-0.16.0.tar.xz
cc:url_aarch64_darwin) → .../zig-aarch64-macos-0.16.0.tar.xz
cc:sha256_* )     → per-platform (re-verify from index.json; or
fetch+parse index.json at obtain time)
cc:container_extract) → no
```

New helpers (paralleling _go_plat / resolve_go / obtain_via_prebuilt):

- `_zig_plat()` — `uname` → zig tuple x86_64-linux / aarch64-linux / x86_64-macos / aarch64-macos; anything else → `EC_UNSUPPORTED` (§11.4.6 honest, §11.4.111 resolve-by-name not ordinal).

- `resolve_cc()` — **resolve-by-CAPABILITY, not mere presence:** take the host `cc/gcc/clang` (absolute, §11.4.111) and `cc_can_link`-probe it (compile+link a trivial exe). ONLY a host compiler that LINKS counts as resolved (this is what makes "gcc present but cannot link" fall through to obtain). On success emit `CC_BIN=<hostcc>`, `CC_KIND=host`, `CC_SOURCE=host-system`.
- `obtain_via_prebuilt_zig()` — download `.tar.xz`, verify sha256 (against the hardcoded pin AND/OR `index.json`), extract to `$LOCAL_PREFIX/zig/`, run `<dir>/zig` version as the anti-bluff proof it executes (§11.4.5), then smoke-test `echo 'int main(){return 0;}' | <dir>/zig cc -x c - -o /tmp/zt && /tmp/zt` — i.e. PROVE the obtained toolchain can LINK before declaring success (never a fake PASS). Emit `CC_BIN=<dir>/zig`, `CC_KIND=zig`, `CC_SOURCE=local-toolchain`.
- In main, handle `kind=toolchain` for `cc` like go: RESOLVE-first (`resolve_cc`), else OBTAIN; write `CC_BIN / CC_KIND / CC_SOURCE` to `resolved.env`; continue.

6.2 obtain_via_source MUST consume the obtained CC

Today `obtain_via_source` picks `cc="$(_first_exe ... cc gcc clang)"` — on a bare host that is the can't-link gcc, so `libevent/ncurses/jemalloc` source builds ALSO fail. Change it to prefer the resolved toolchain: if `CC_KIND=zig`, build each dep with `CC="<CC_BIN> cc"` (and export `CC / ./configure CC="$CC"`). **Order matters:** obtain `cc` FIRST. `setup.sh` Step 1b becomes `DEPS="cc libevent ncurses jemalloc"` (`cc` leads) so the toolchain is ready before any source build runs.

6.3 scripts/build_native.sh (Linux branch) — consume CC + build from the release tarball

- Source `resolved.env` (already done at line 188) and, when `CC_KIND=zig`, export `CC="$CC_BIN cc"` (+ export `YACC=true`).
- **Build source decision:** if `tmux/configure` is absent AND no host autotools (the bare-host case), extract the sha256-pinned 3.6a release tarball into a build dir (e.g. `$REPO_ROOT/.local-deps/<plat>/tmux-src/`), touch `cmd-parse.c`, and run `./configure` THERE. Otherwise keep the existing submodule-working-tree path unchanged (zero regression on full hosts).
- `libevent/ncurses` wiring is ALREADY correct (lines 174–222 set `LE_CPPFLAGS/LE_LDFLAGS/NC_CPPFLAGS/NC_LDFLAGS + PKG_CONFIG_PATH` from `resolved.env`). With `pkg-config` absent these `-I/-L` flags drive the `AC_SEARCH_LIBS` fallback (§1.6); with `pkg-config` present, `PKG_CONFIG_PATH` is used — both work.
- Pass `CC` into `configure`: `CC="$CC" CFLAGS=... LDFLAGS=... ./configure ....`

6.4 scripts/setup.sh — wire the toolchain obtain into the preflight

setup.sh already has `cc_can_link + _native_build_preflight` (lines 39-180) and the auto-install path (currently distro package manager → needs sudo). Add a ROOT-FREE branch: when `cc_can_link` fails, run the `cc` toolchain obtain (`DEPS=cc obtain_local_deps.sh`), re-probe with `CC="$CC_BIN cc"`, and proceed if that links — BEFORE falling back to the sudo `install_deps.sh` advice. This makes the existing "C compiler cannot create executables" path self-heal without root.

6.5 Four-layer coverage (per §11.4.4(b)) the implementation stream owes

- pre-build gate: registry has `cc` entry + per-platform url/sha + `EC_UNSUPPORTED` for other arches; `build_native` exports `CC/YACC`.
 - runtime test: extend `scripts/tests/70_native_fallback_cc_link.sh` (already exercises the preflight in library mode) + add a `tests/NN_zig_toolchain.sh` that obtains zig, links a probe, and asserts `CC_KIND/CC_SOURCE`.
 - §1.1 paired mutation: strip the `cc` registry branch / the `YACC=true` export → the bare-host build test FAILs.
 - the §5 live test below is the user-visible proof.
-

7. HONEST risks / boundaries — what MUST be live-tested before any "works" claim

1. **zig cc builds tmux end-to-end** — UNPROVEN here. Must live-run a full `zig cc` build of `libevent + ncurses + jemalloc + tmux` and RUN the resulting `tmux` (`tmux -V`, start a session). The §1.5/§1.6 bypasses are verified at the configure level; the `COMPILE+LINK` of all four with zig is not.
2. **-D_FORTIFY_SOURCE=2 / -fstack-protector-strong / -Wl,-z,relro,-z,now under zig clang+lld** — verify no hard error; adjust the glibc target floor if FORTIFY headers complain.
3. **ncurses source build under zig cc** — the most complex dep build; live-test.
4. **zig sha256 values** (§2.3) — PROVISIONAL; re-confirm against `index.json` (+ ideally `.minisig`) at commit time. Do NOT ship the table values unverified.
5. **.tar.xz extraction** — depends on host `xz`; present here, but a truly `xz-less` host fails. Honest EC + message (do not fake).
6. **make residual** — assumed present; if a `make-less` host appears it is a separate follow-on obtain (out of scope here).

7. **macOS zig-cc fallback** — UNCONFIRMED (§5); no claim until proven on a Mac.
 8. **glibc native-detection** — verify the zig-native binary actually runs on the host (it should, since build host == run host); pin a glibc floor only if detection misbehaves.
-

8. Test plan — fully-autonomous, root-free, on a REAL host

Goal: prove a bare-host build using ONLY the obtained toolchain + local libs. This ALT host HAS a working toolchain, so the test must **NEUTER/hide the system toolchain** to simulate the bare host (no sudo, no system mutation — use a sanitised PATH + redirected absolute lookups), then build + run.

1. **Baseline capture (anti-bluff)**: record `gcc -dumpmachine`, `gcc <tiny.c>` links OK, `tmux -V` of any existing build — proves the host IS capable, so a later success is attributable to the obtained toolchain, not the host.
2. **Neuter the system toolchain** in a subshell (no root): run the whole build under a sanitised environment where the system C toolchain is unreachable — e.g. `env -i HOME=$HOME PATH=<dir-with-only-make/tar/xz/curl/sha256sum/sh>` (a shim dir that DELIBERATELY omits `gcc/cc/clang/autoconf/automake/bison/pkg-config`), plus point the absolute-path resolvers at non-existent paths. Verify inside the subshell: `command -v gcc` empty, `cc -o /tmp/x x.c` fails with the very "cannot create executables"-class error. (Captured = the proof the simulation is real.)
3. **Run the obtain + build** inside that neutered subshell: `FORCE_OBTAIN=1 DEPS="cc libevent ncurses jemalloc" bash scripts/obtain_local_deps.sh` then `bash scripts/build_native.sh`. Expect: zig obtained, `CC_KIND=zig`, `libevent/ncurses/jemalloc` source-built with zig cc, tmux configured from the release tarball (`YACC=true`, `AC_SEARCH_LIBS` path), compiled, linked.
4. **User-visible proof (§11.4.5)**: `tmux/build/bin/tmux -V` prints `tmux 3.6a`; start a detached session, `send-keys/capture-pane`, confirm live output; `file tmux` shows a host-runnable ELF; `ldd tmux` resolves (or the jemalloc preload path works) — captured under `docs/qa/<run-id>/`.
5. **Determinism (§11.4.50)**: repeat 3 the build `N=3` times → identical success + identical `tmux -V`.
6. **Negative/regression guard (§11.4.115)**: `RED_MODE=1` = run the build in the neutered subshell WITHOUT the toolchain obtain → MUST fail at the can't-link step (proves the test exercises the real gap); `RED_MODE=0` = with the obtain → MUST pass. Same source, polarity switch.
7. **Full-host regression**: on the unmodified host (toolchain present), `setup.sh` must take the UNCHANGED host path (host

cc resolves, no zig obtained) — proves zero regression for normal hosts.

A "works" claim is earned ONLY after steps 3-6 produce captured evidence on a real host; until then every assertion in §2.4/§2.5/§7 is explicitly UNCONFIRMED.

Sources verified 2026-06-29

- Zig downloads (version, URL pattern, JSON index, minisign key): <https://ziglang.org/download/> — fetched 2026-06-29 (latest stable 0.16.0, released 2026-04-13).
- Zig release JSON index (per-platform tarball/shasum/size schema): <https://ziglang.org/download/index.json> — fetched 2026-06-29.
- Zig signature/verification guidance + minisign public key: <https://ziglang.org/download/> ("validate the minisign signature for every tarball ... public key RWSGOq2NVecA2UPNdBUZykf1CCb147pkmdtYxgb3Ti+JO/wCYvhbAb/U").
- zig cc as a drop-in gcc/clang replacement (mechanics, glibc targeting): <https://andrewkelley.me/post/zig-cc-powerful-drop-in-replacement-gcc-clang.html> — accessed 2026-06-29.
- zig cc + autotools pitfall (CC=zig vs CC='zig cc', -o parse failure): <https://github.com/ziglang/zig/issues/12413> — accessed 2026-06-29.
- zig cc + autotools/glibc-version cross-build precedent (./configure ... CC='zig cc --target=...' --host=...): <https://richiejp.com/zig-cross-compile-ltp-ltx-linux> — accessed 2026-06-29.
- zig glibc version support / -target ...-gnu.2.x: <https://github.com/ziglang/zig/blob/master/lib/libc/glibc/README.md>
 - <https://ziglang.org/learn/overview/> — accessed 2026-06-29.
- tmux 3.6a release tarball (ships pre-generated configure/aclocal.m4/Makefile.in/ cmd-parse.c; sha256 b6d8d9c76585db8ef5fa00d4931902fa4b8cbe8166f528f44fc403961a3f3759): <https://github.com/tmux/tmux/releases/download/3.6a/tmux-3.6a.tar.gz> — downloaded + inspected 2026-06-29.
- Bootlin prebuilt toolchains (alternative, Linux-only): <https://toolchains.bootlin.com> — referenced 2026-06-29.
- cargo-zigbuild (corroborates zig cc glibc-version targeting in production): <https://crates.io/crates/cargo-zigbuild> — referenced 2026-06-29.

(Repo-internal FACTS captured live 2026-06-29: git submodule status tmux, git -C tmux ls-files, tmux/configure.ac, tmux/autogen.sh, the extracted release-tarball configure lines 4578/4588/4619/6081/5956, xz/tar versions, scripts/obtain_local_deps.sh, scripts/build_native.sh, scripts/setup.sh.)